Module 5

Virtual Memory and File Management

Dr. Ngangbam Indrason
Assistant Professor - SCOPE
VIT-AP University, Amaravati

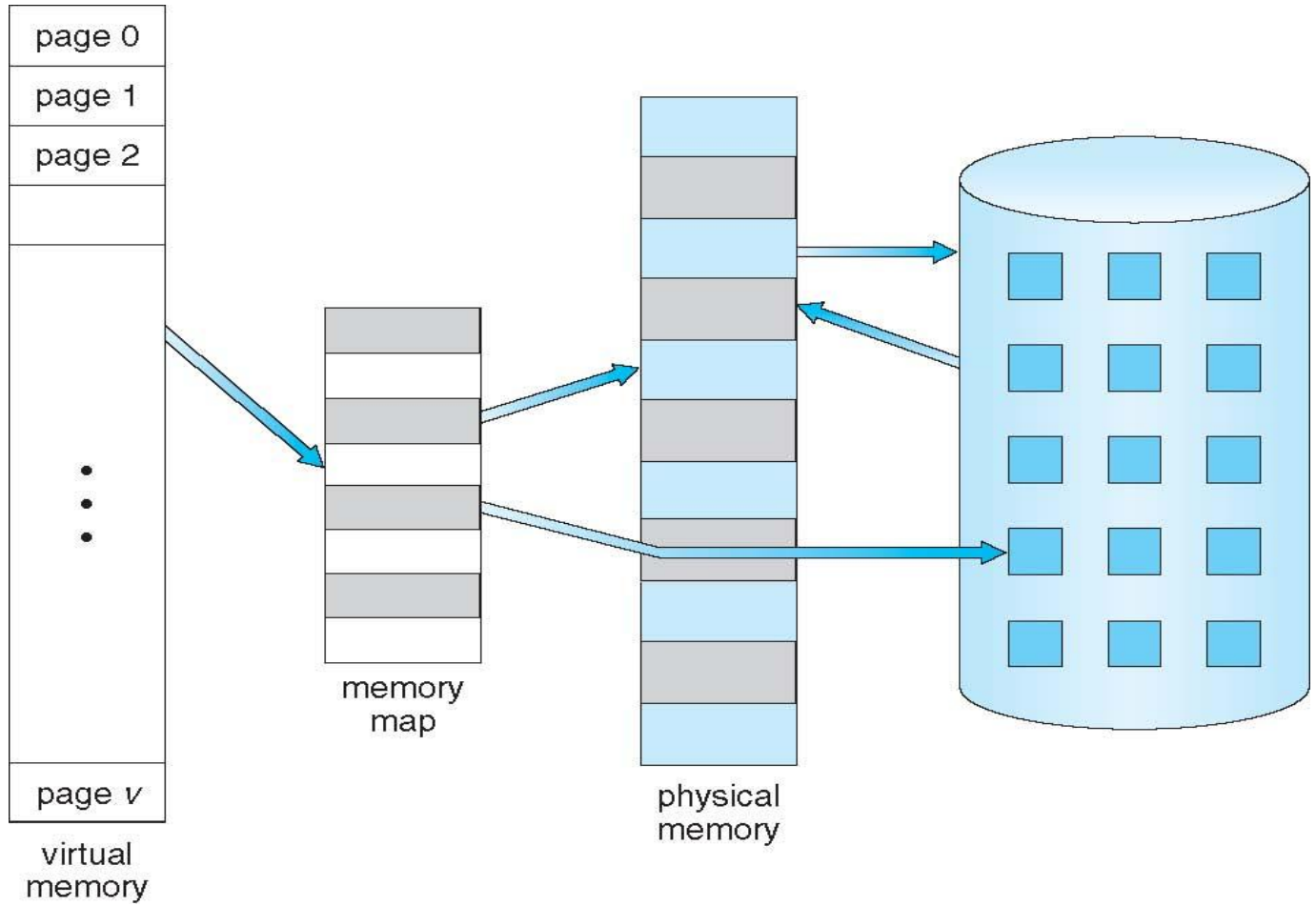
Virtual Memory

- Virtual memory – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes

Virtual Memory

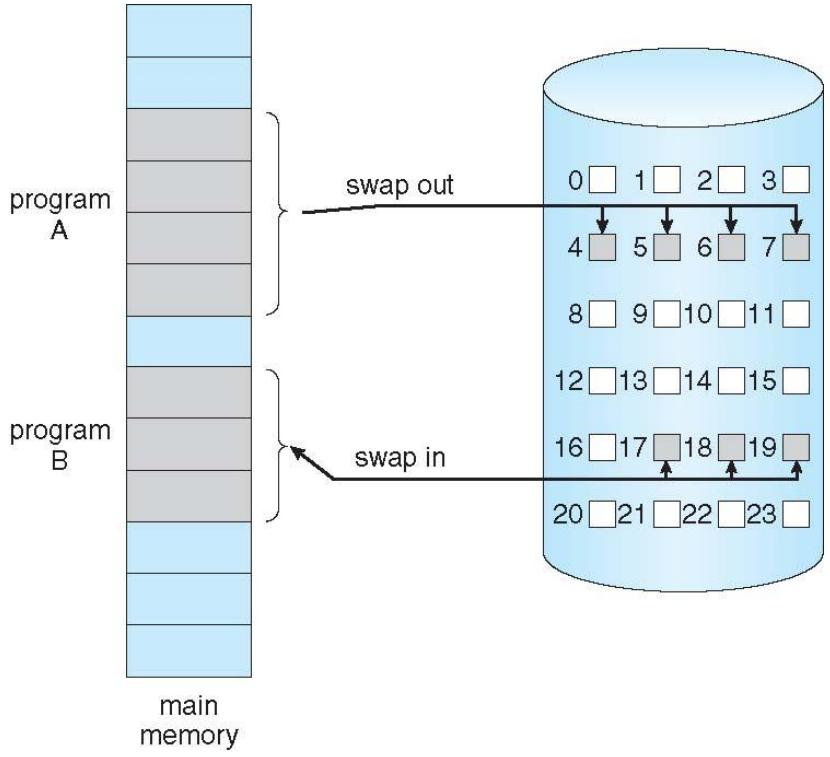
- Virtual address space – logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

Virtual Memory That is Larger Than Physical Memory

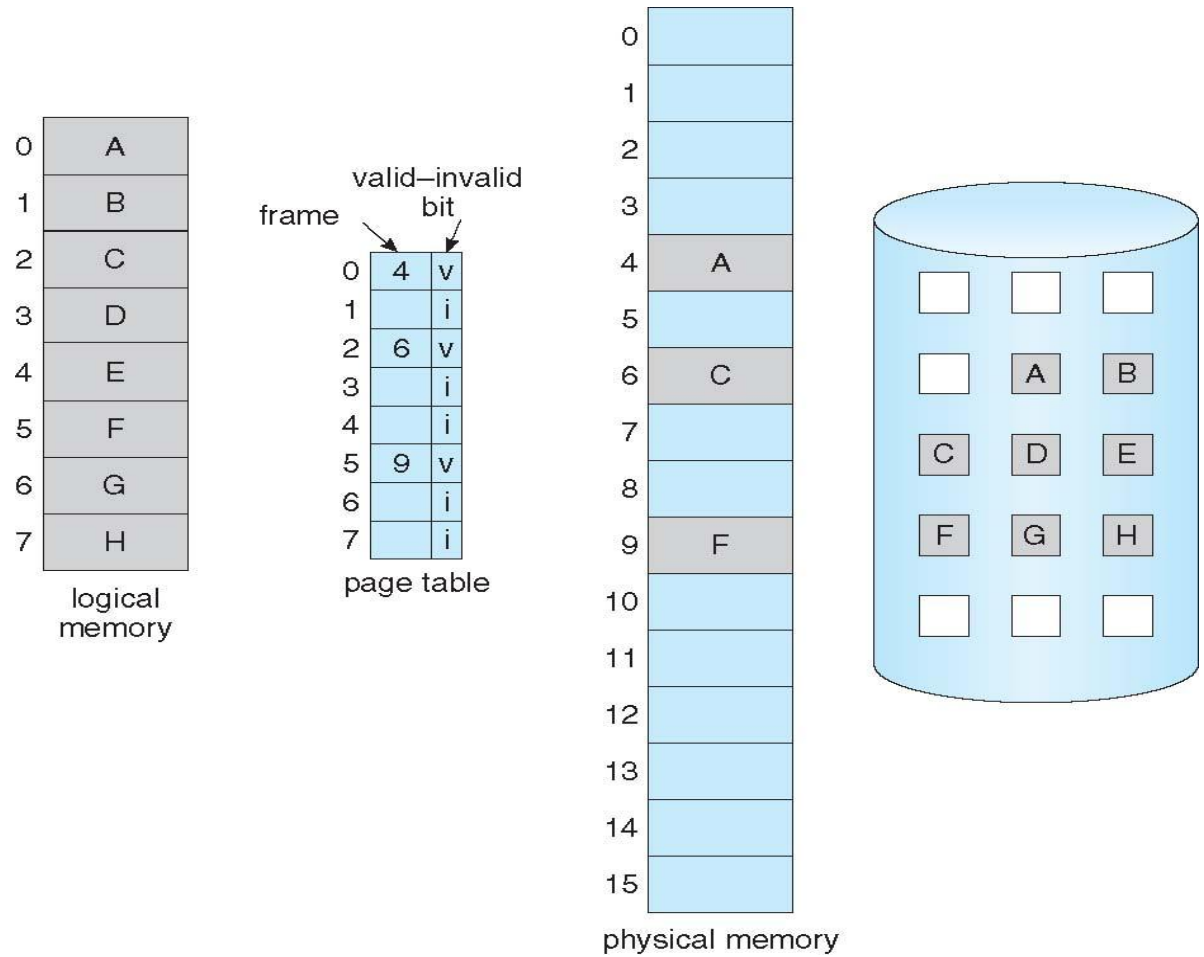


Demand Paging

- Demand paging is a virtual memory implementation where pages are only loaded into memory when they are actually needed (i.e., when a reference is made to a location on the page).
  - It's similar to a paging system with swapping.
  - It is supported by the valid-invalid bit in the Page Table Entry (PTE).
    - Valid (v): The page is currently in memory.
    - Invalid (i): The page is either not valid (outside the process's address space) or is valid but currently on disk (in the swap space).



Page Table When Some Pages Are Not in Main Memory

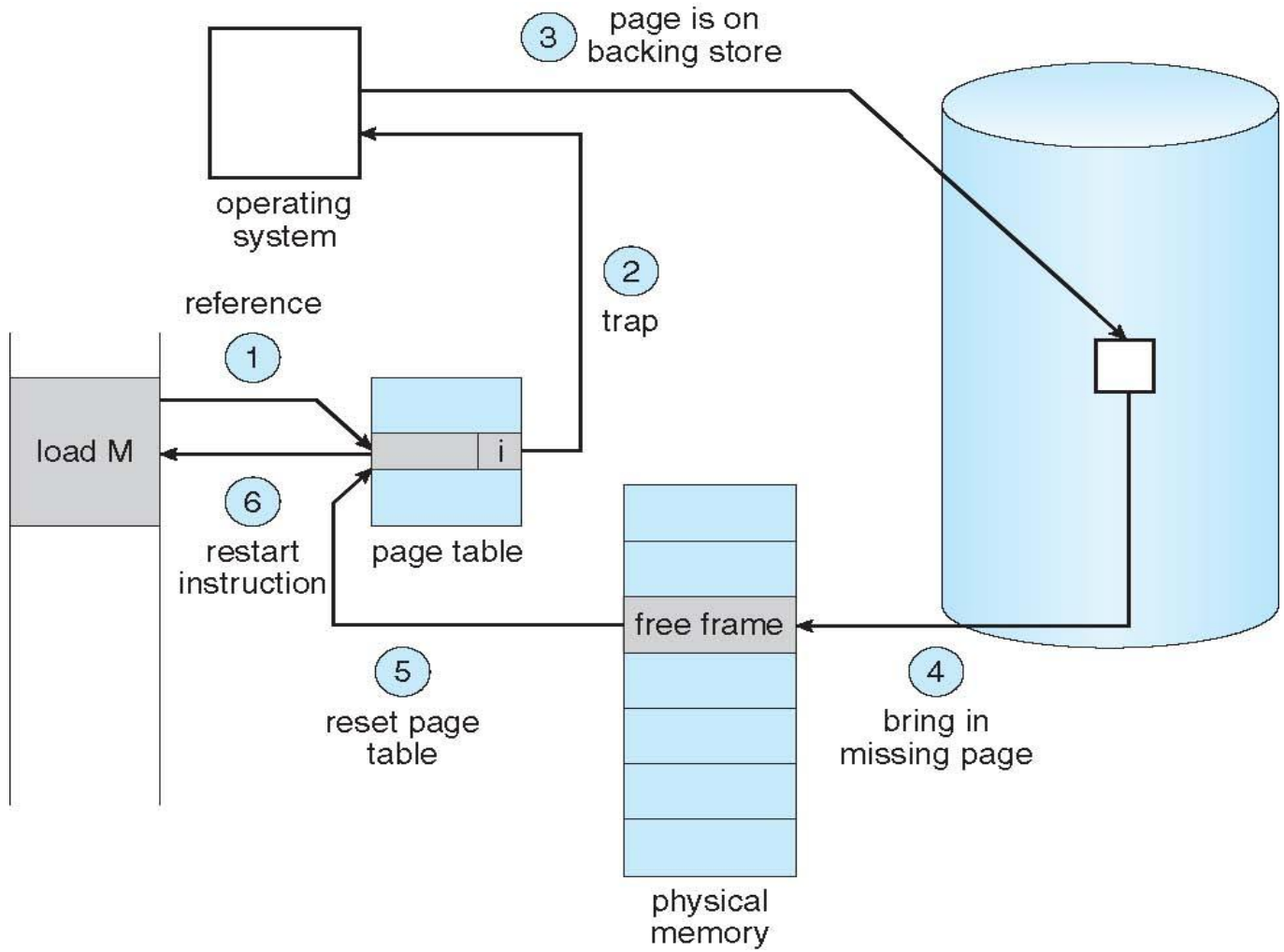


Page Fault

- A page fault is an interrupt that occurs when a process tries to access a page that is currently not in physical memory (the PTE valid/invalid bit is set to 'invalid' for that page).

1. If there is a reference to a page, first reference to that page will trap to operating system: page fault
2. Operating system looks at another table to decide:
   - Invalid reference → abort
   - Just not in memory
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory
6. Set validation bit = v
7. Restart the instruction that caused the page fault

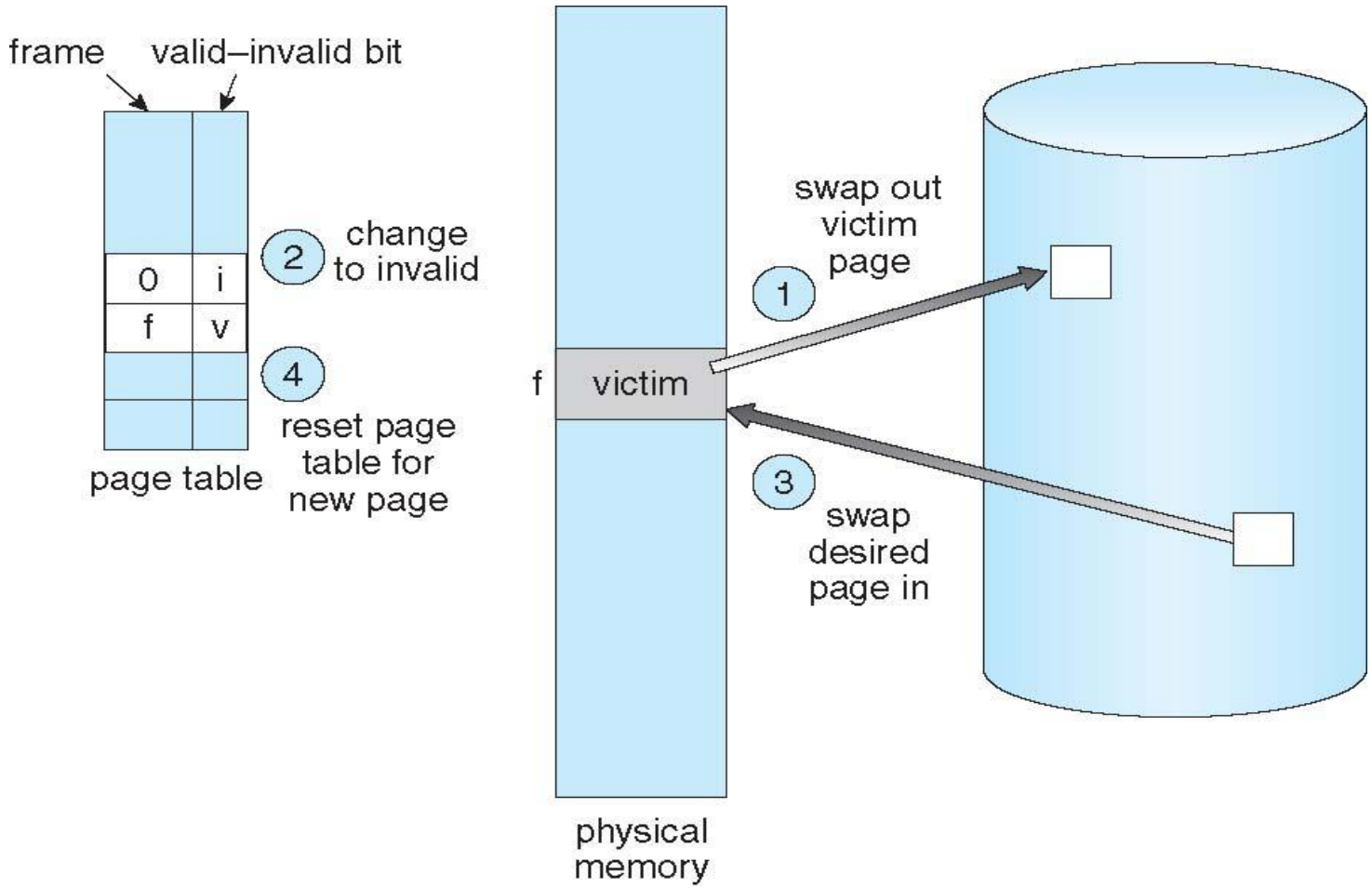
Steps in Handling a Page Fault



Page Replacement Algorithms

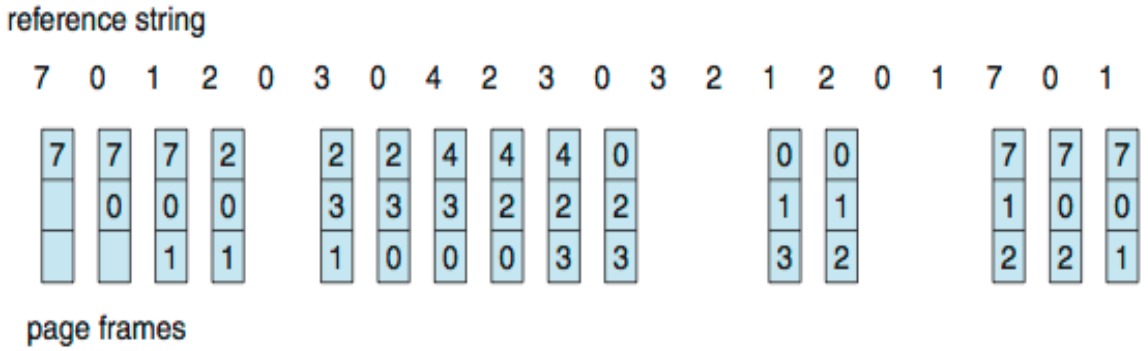
- When a page fault occurs and all memory frames are occupied, a page replacement algorithm is used to select a "victim" frame to be swapped out to the disk to free up a frame for the new page. The goal is to minimize the page-fault rate.

Page Replacement Algorithms



First-In-First-Out (FIFO) Algorithm

- Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)

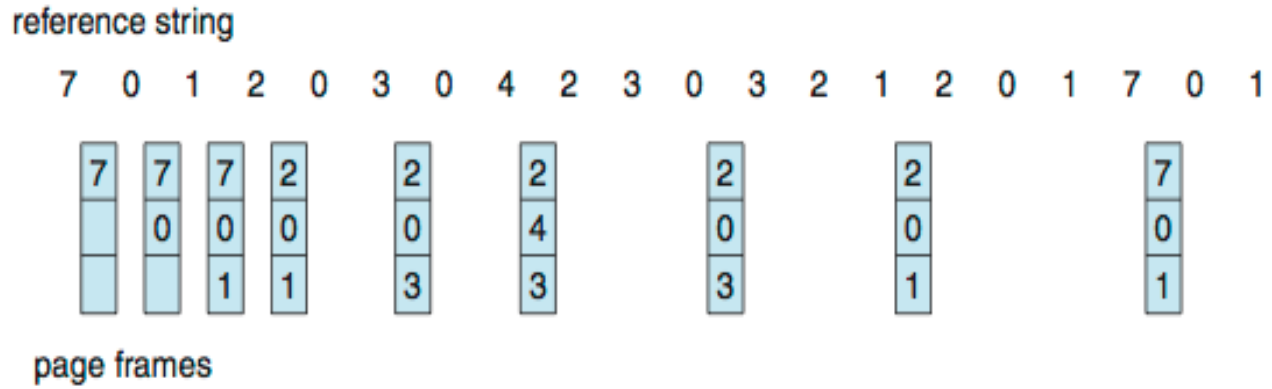


15 page faults

- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
  - Adding more frames can cause more page faults!
    - Belady's Anomaly
- How to track ages of pages?
  - Just use a FIFO queue

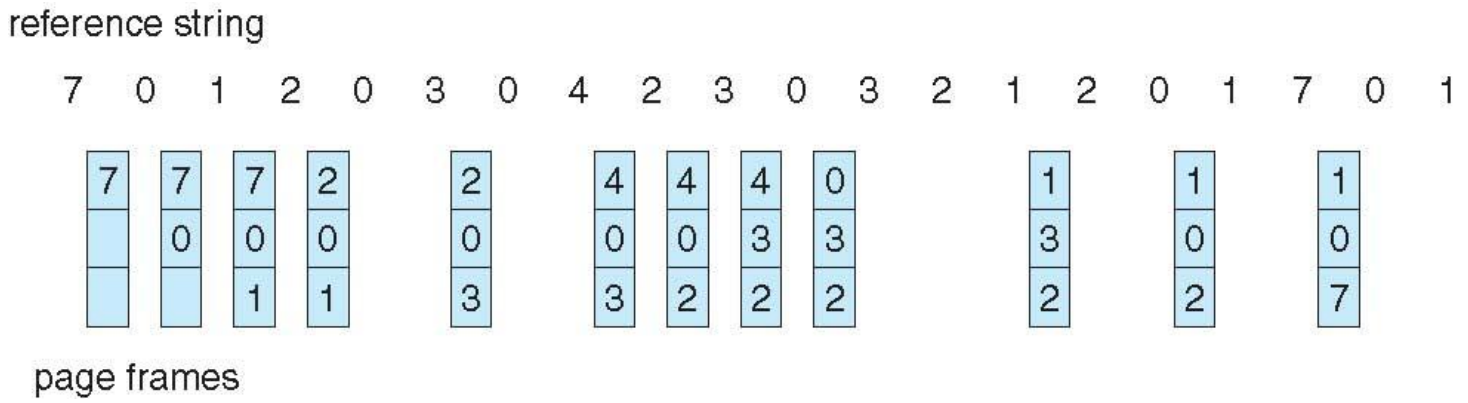
Optimal Algorithm

- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs



Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page



- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

LRU Algorithm

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value: Search through table needed
- Stack implementation
  - Keep a stack of page numbers in a double link form:
  - Page referenced: move it to the top, requires 6 pointers to be changed
  - But each update more expensive
  - No search for replacement
- LRU and OPT are cases of stack algorithms that don’t have Belady’s Anomaly

File Concept

- Contiguous logical address space
- Types:
  - Data
    - numeric
    - character
    - binary
  - Program
- Contents defined by file's creator
  - Many types
    - Consider **text file, source file, executable file**

File Attributes

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk
- Many variations, including extended file attributes such as file checksum
- Information kept in the directory structure

File Operations

- File is an **abstract data type**
- **Create**
- **Write –** at **write pointer** location
- **Read –** at **read pointer** location
- **Reposition within file - seek**
- **Delete**
- **Truncate**
- ***Open(F_i)*** – search the directory structure on disk for entry ***F_i***, and move the content of entry to memory
- ***Close (F_i)*** – move the content of entry ***F_i*** in memory to directory structure on disk

File Structure

- None - sequence of words, bytes
- Simple record structure
  - Lines
  - Fixed length
  - Variable length
- Complex Structures
  - Formatted document
  - Relocatable load file
- Who decides:
  - Operating system
  - Program

Directory Structure

- The directory structure is the organization of files on the storage device. It dictates how users access files and how the OS manages them.
- A collection of nodes containing information about all files
- Both the directory structure and the files reside on disk

Operations Performed on Directory

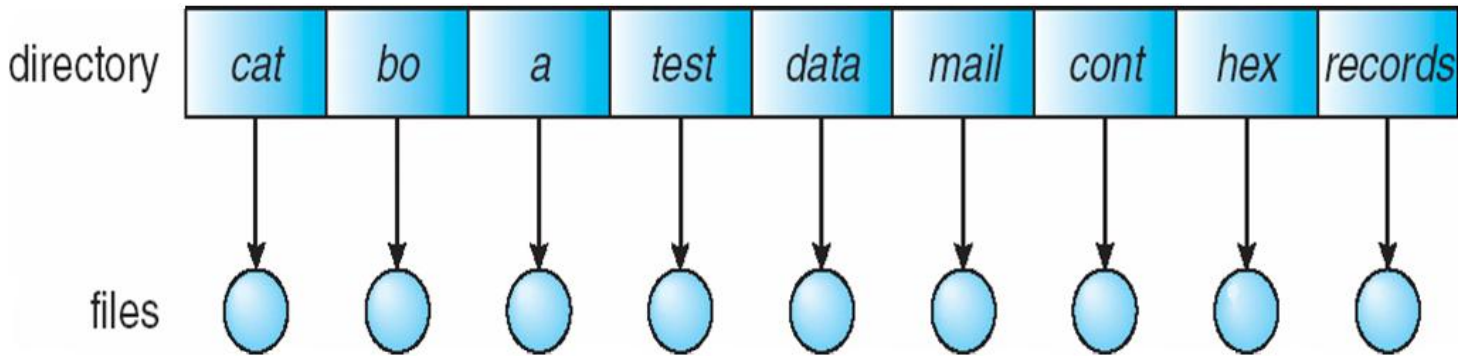
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

Directory Organization

- The directory is organized logically to obtain
  - Efficiency – locating a file quickly
  - Naming – convenient to users
    - Two users can have same name for different files
    - The same file can have several different names
  - Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, …)

Single-Level Directory

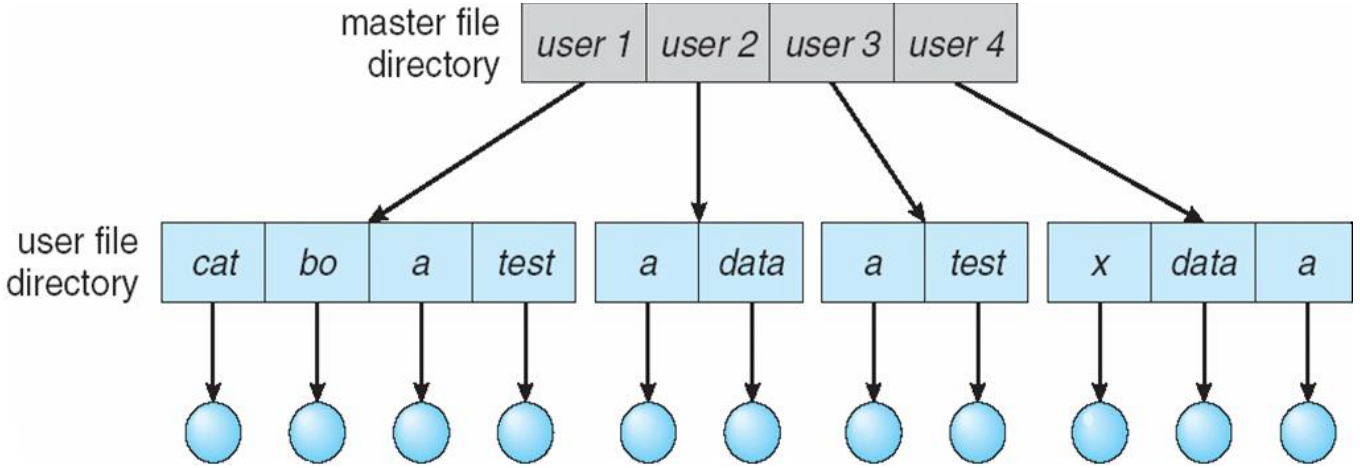
- A single directory for all users



- Naming problem
- Grouping problem

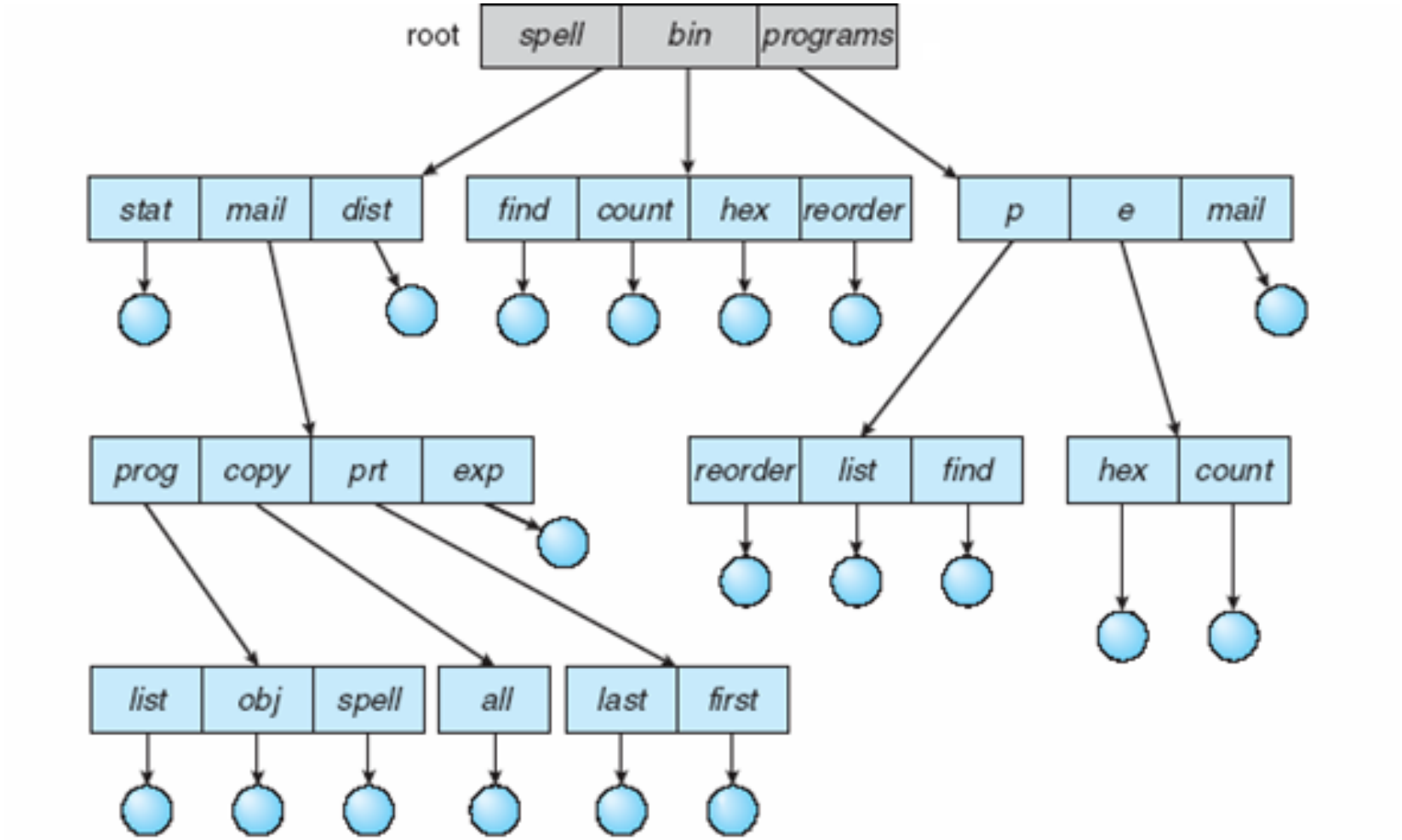
Two-Level Directory

- Separate directory for each user



- Can have the same file name for different user
- Efficient searching
- No grouping capability

Tree-Structured Directories



Tree-Structured Directories

- Efficient searching
- Grouping Capability

- Current directory (working directory)
  - **`cd /spell/mail/prog`**
  - **`type list`**

Tree-Structured Directories

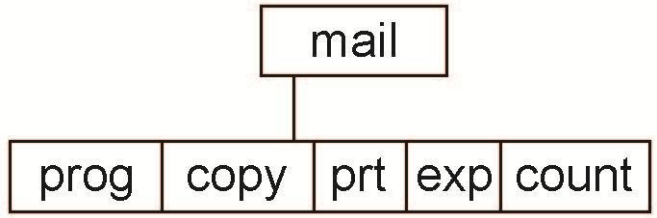
- **Absolute** or **relative** path name
- Creating a new file is done in current directory
- Delete a file

  rm <file-name>
- Creating a new subdirectory is done in current directory

  mkdir <dir-name>

  Example: if in current directory **/mail**

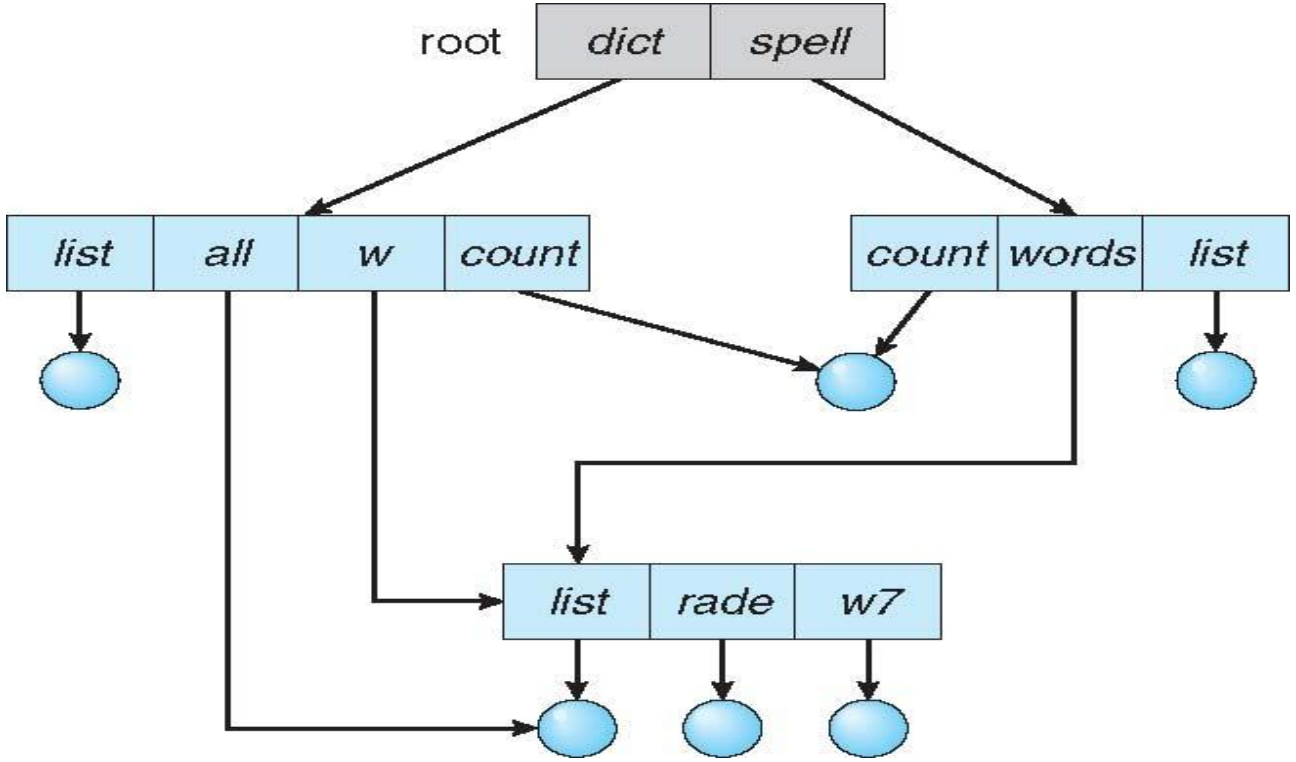
  mkdir count



- Deleting "mail" ⇒ deleting the entire subtree rooted by "mail"

Acyclic-Graph Directories

- A variation of the tree structure that allows **file sharing.**
- Have shared subdirectories and files



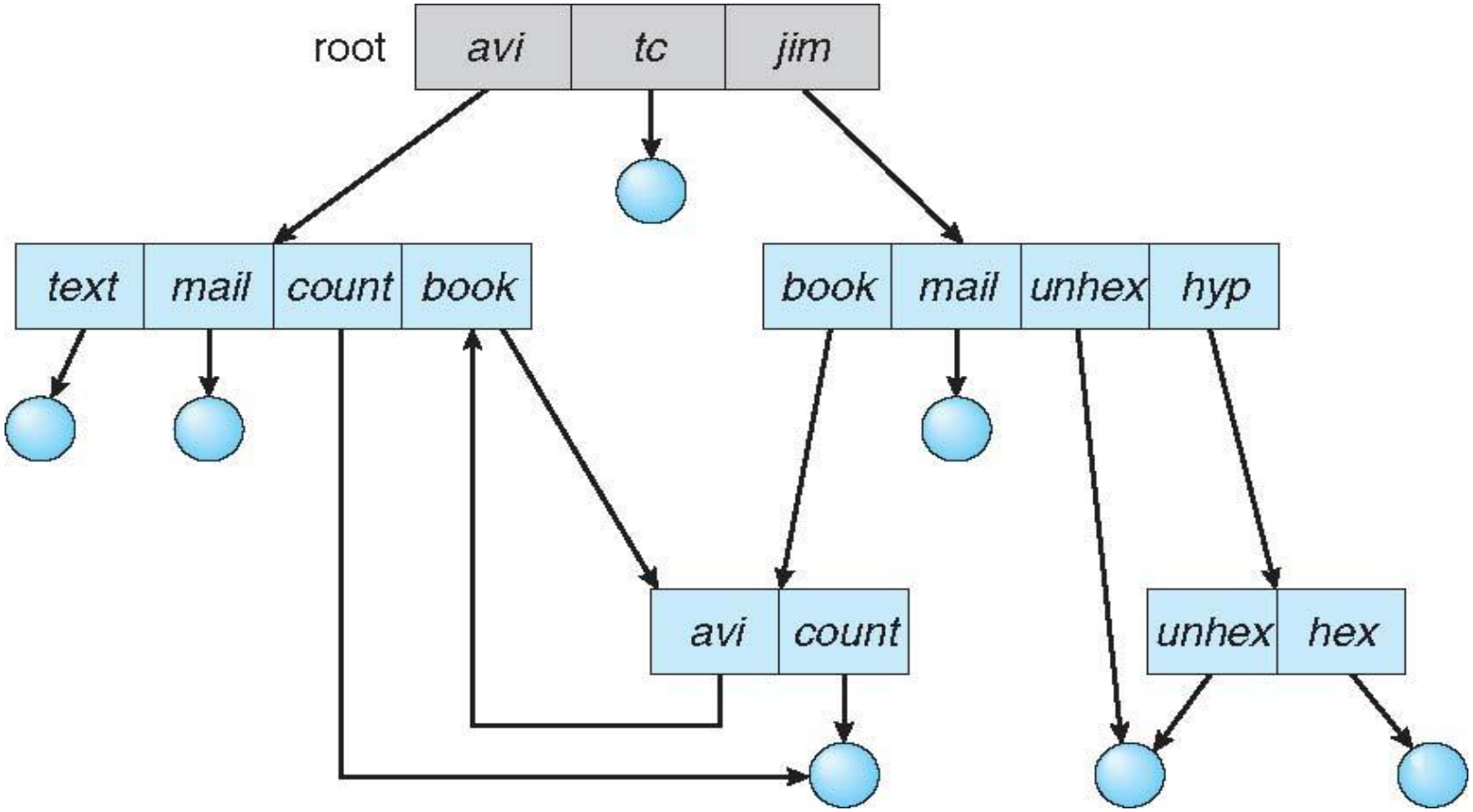
Acyclic-Graph Directories

- Two different names (aliasing)
- If ***dict*** deletes ***list*** $\Rightarrow$ dangling pointer

  Solutions:
  - Backpointers, so we can delete all pointers
    Variable size records a problem
  - Backpointers using a daisy chain organization
  - Entry-hold-count solution
- New directory entry type
  - **Link** – another name (pointer) to an existing file
  - **Resolve the link** – follow pointer to locate the file

General Graph Directory

- Allows arbitrary links between directories, creating cycles (loops).



General Graph Directory

- How do we guarantee no cycles?
  - Allow only links to file not subdirectories
  - **Garbage collection**
  - Every time a new link is added use a cycle detection algorithm to determine whether it is OK

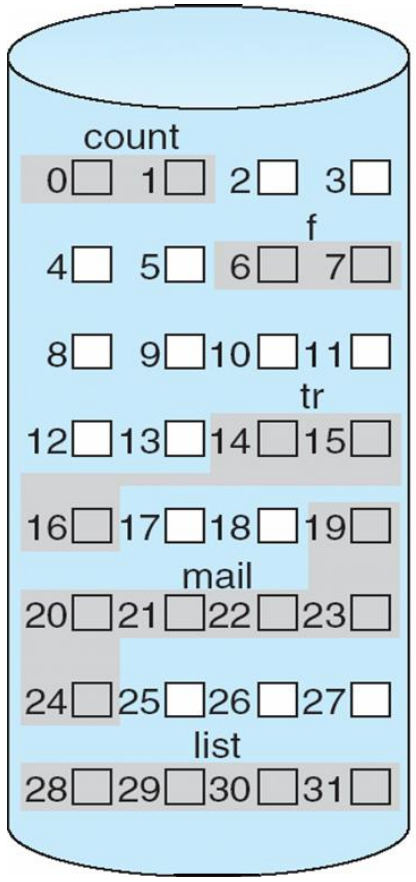
Directory Structures

Structure	Description	Key Feature
Single-Level	All files in one directory.	Simplest, but poor scalability and suffers from naming conflicts.
Two-Level	Master directory points to separate directories for each user.	Solves name conflicts between users; isolation is the default.
Tree-Structured	A hierarchical structure with a single root, forming an inverted tree.	Most common (used by Windows, Linux); allows logical grouping and pathnames.
Acyclic-Graph	Tree structure with added **links** (shortcuts) to allow files to be shared from multiple locations without duplication.	Allows non-duplicative file sharing but prohibits loops.
General Graph	Allows loops (cycles) in the directory structure.	Most flexible, but requires complex algorithms to prevent infinite loops during traversal or search.

Allocation Methods - Contiguous

- File allocation methods determine how the blocks belonging to a file are stored and organized on the secondary storage device (disk).
- **Contiguous allocation** – each file occupies set of contiguous blocks
  - Best performance in most cases
  - Simple – only starting location (block #) and length (number of blocks) are required
  - Problems include finding space for file, knowing file size, external fragmentation, need for **compaction off-line** (**downtime**) or **on-line**

Contiguous Allocation



directory

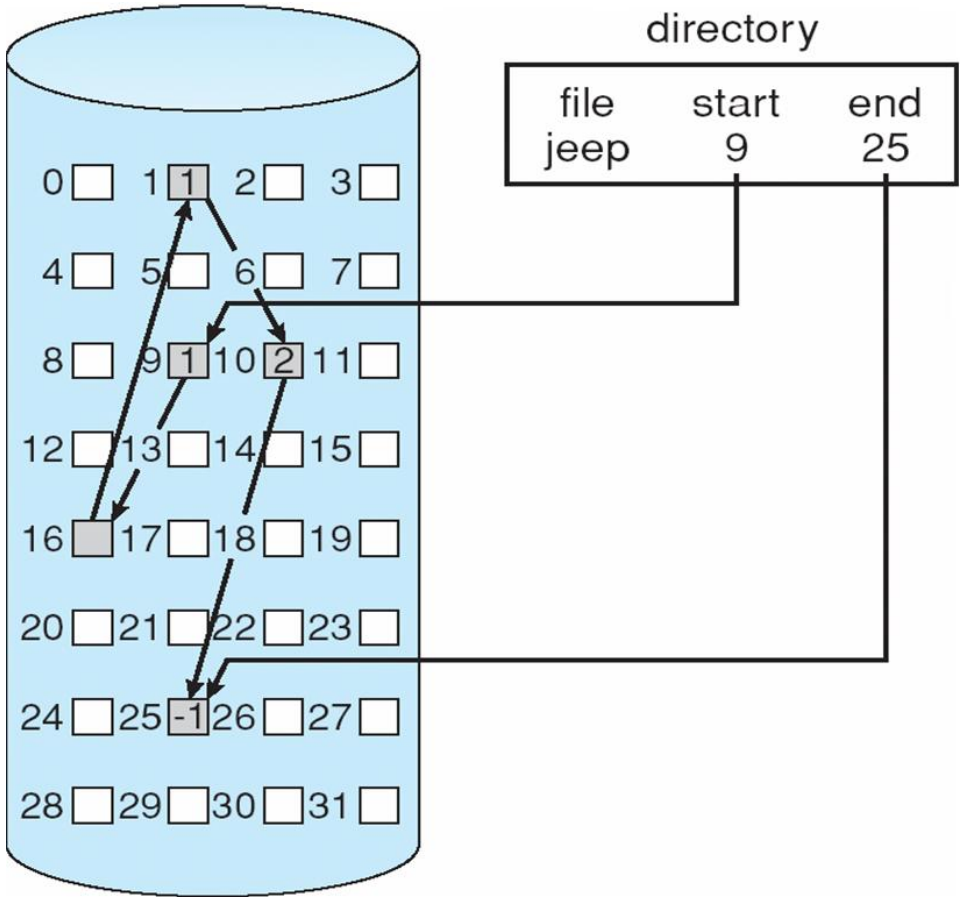
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Allocation Methods - Linked

- **Linked allocation** – each file a linked list of blocks
  - File ends at null pointer
  - No external fragmentation
  - Each block contains pointer to next block
  - No compaction, external fragmentation
  - Free space management system called when new block needed
  - Improve efficiency by clustering blocks into groups but increases internal fragmentation
  - Reliability can be a problem
  - Locating a block can take many I/Os and disk seeks

Allocation Methods – Linked

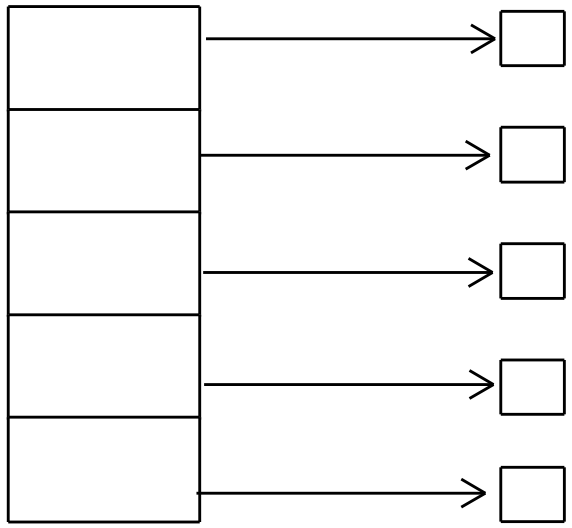
- FAT (File Allocation Table) variation
  - Beginning of volume has table, indexed by block number
  - Much like a linked list, but faster on disk and cacheable
  - New block allocation simple



Allocation Methods - Indexed

- **Indexed allocation**
  - Each file has its own **index block**(s) of pointers to its data blocks

- Logical view



Indexed Allocation

- Need index table
- Random access
- Dynamic access without external fragmentation, but have overhead of index block
- Mapping from logical to physical in a file of maximum size of 256K bytes and block size of 512 bytes. We need only 1 block for index table

Combined (Linked + Indexed) Allocation

- This method addresses the size limitations of simple indexed allocation by using schemes like those found in the **UNIX/Linux file system (UFS/ext)**.
- **Method:** A file's metadata (**inode**) contains a mixed scheme of pointers:
  - **Direct Pointers:** The first few pointers in the index structure (typically 12 or 15) point directly to the file's first data blocks. This handles small files efficiently with a single read.
  - **Indirect Pointers:** Used for medium and large files:
    - **Single Indirect:** One pointer in the index block points to a **second block** (the indirect block), which in turn contains pointers to data blocks.
    - **Double Indirect:** One pointer points to a block that contains pointers to **single indirect blocks**.
    - **Triple Indirect:** One pointer points to a block that contains pointers to **double indirect blocks**.
- **Benefit:** This provides fast access for small files (via direct pointers) while allowing for files of truly **massive size** (via triple indirect pointers)

End of Module 5